

Defense in Depth: Layers, Not Walls

The onion principle

Any single control can be bypassed. A firewall can be RULEd around; a patch can be missed; an antivirus can be evaded. Defense in depth accepts imperfection and layers **independent** controls so that failure of any one layer does not equal compromise.

Attacker must defeat (in order):

perimeter firewall -> host firewall -> app hardening

-> least privilege -> EDR -> encryption at rest

-> monitoring & detection -> IR team

Independence is the key

Layers must not share the same flaw:

- Bad: firewall AND switch ACL both use the same stale IP allow-list
- Good: perimeter firewall + host firewalls patched by separate teams + app-level auth + encryption

If every layer trusts the same single source, you have one wall, not many.

The tools per layer

Layer	Typical controls
Network	segmentation, ZTNA, egress filtering, IDS/IPS
Host	hardening, EDR, application allow-listing
App	secure coding, WAF, input validation, authN/authZ
Data	encryption, DLP, backup

Layer	Typical controls
People	training, least privilege, separation of duties

Fail-closed vs fail-open

Design decisions: if a control breaks, does access become **denied** (fail-closed – safer but availability hits) or **allowed** (fail-open – convenient but dangerous)? Authentication should fail-closed; a load balancer health check may fail-open.

Classic failure: a misconfigured "internal only" service that authenticates via network presence. Defense in depth says such a service still requires app-layer auth – because that network boundary WILL someday be pierced.

[← Module start](#)

[Zero Trust: Never Trust, Always...](#)